Cloud PKI Integration with Intune (Certificate Management Modernization) — Government (GCC High)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Cloud PKI Integration with Intune (Government / GCC High) 4

The problem . . . 4

What this solution does . . . 4

The GCC High differences that matter (doc-confirmed) . . . 4

Business outcome . . . 4

Solution Overview — Cloud PKI Integration with Intune (Government / GCC High) 5

Read this first — Cloud PKI availability in GCC High . . . 5

What this solution delivers . . . 5

Why GCC High is a different solution, not a re-login . . . 6

The certificate model (how the pieces fit) . . . 6

Commercial vs GCC High — what's different . . . 7

The standard deployment model . . . 8

What's in the box . . . 8

Requirements — Cloud PKI Integration with Intune (Government / GCC High) 9

R0. Confirm Cloud PKI is out of scope for now (parity acknowledgement) . . . 9

R1. A Microsoft 365 Government – GCC High tenant . . . 9

R2. Government Intune licensing (G3 / G5 with Intune Plan 1) . . . 9

R3. Intune administrative roles . . . 10

R4. An issuing Certification Authority (AD CS) reachable on-network . . . 10

R5. NDES (for SCEP with a Microsoft CA) . . . 10

R6. The Certificate Connector for Microsoft Intune (installed from the GCC High portal) . . . 10

R7. (If using a partner CA) partner-CA support for GCC High . . . 11

R8. Network / filtering allow-list for GCC High endpoints . . . 11

R9. Enrolled devices on a supported OS, and a pilot scope + change control . . . 11

R10. Parity confirmation for the certificate features you intend to use . . . 11

Implementation — Cloud PKI Integration with Intune (Government / GCC High) 12

Phase 0 — Assess (gate before you build) . . . 12

Phase 1 — Publish the trust chain (trusted certificate profile) . . . 13

Phase 2 — SCEP certificate profile (dynamic per-device issuance) . . . 13

Phase 3 — PKCS certificate profile (alternative / user certificates) . . . 14

Phase 4 — Consume the certificate (Wi-Fi / VPN) and prove renewal/revocation . . . 15

Phase 5 — Pilot → Validate → Expand . . . 15

Testing — Cloud PKI Integration with Intune (Government / GCC High) 16

Gate 0 — Sovereign-cloud sanity (must pass first) . . . 16

Gate 1 — Cloud PKI parity acknowledged (not assumed) . . . 16

Gate 2 — Licensing & roles present . . . 16
Gate 3 — CA + NDES + connector healthy . . . 16
Gate 4 — Trust chain deployed . . . 17
Gate 5 — SCEP certificate issued . . . 17
Gate 6 — PKCS certificate issued (if in scope) . . . 17
Gate 7 — Certificate consumed (Wi-Fi / VPN authenticates) . . . 17
Gate 8 — Renewal works . . . 17
Gate 9 — Revocation works . . . 18
Gate 10 — Script validation clean . . . 18
Troubleshooting — Cloud PKI Integration with Intune (Government / GCC High) 18
A. GCC-High-specific gotchas . . . 18
B. Certificate Connector not healthy . . . 19
C. Trust chain not on the device . . . 19
D. SCEP certificate not issued . . . 20
E. PKCS certificate not issued . . . 20
F. Wi-Fi / VPN authentication fails despite a valid certificate . . . 20
G. Renewal not firing . . . 20
H. Revocation not taking effect . . . 20
I. Scripts fail or report nothing . . . 21
Best Practices — Cloud PKI Integration with Intune (Government / GCC High) 21
Sovereign-cloud discipline (GCC High) . . . 21
Cloud PKI specifically . . . 21
Certificate architecture (connector model) . . . 22
Connector & NDES health . . . 22
Rollout & operations . . . 22
Deliverables Summary — Cloud PKI Integration with Intune (Government / GCC High) 23
Documents . . . 23
Scripts (PowerShell, Microsoft Graph SDK, **US Gov endpoints**) . . . 24
Named objects (created and reused verbatim) . . . 25
Outcome . . . 25
Appendix A — PowerShell Scripts 26

Executive Summary — Cloud PKI Integration with Intune (Government / GCC High)

The problem

Certificate management in **GCC High is not the Commercial product with a different sign-in.** The admin portals are separate US sovereign-cloud endpoints (`intune.microsoft.us`, `login.microsoftonline.us`), and — decisively for this topic — **Microsoft Cloud PKI, the marquee Commercial capability for modernizing certificate management, is *not available* in GCC High** and **is not planned for DoD**. A government tenant configured from a Commercial "enable Cloud PKI" runbook silently fails: the capability simply is not present.

What this solution does

It deploys a **managed certificate program on GCC High correctly**, using the **supported** government path: a **trusted-root trust chain** plus **SCEP and PKCS certificate profiles** delivered by the **Intune Certificate Connector**, anchored to your own **AD CS** certification authority — the right portals, the right endpoints, and the Cloud-PKI parity gap called out **up front and at every step where it matters**. It follows a repeatable **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model with pass/fail gates, so certificates are issued, trusted, consumed (Wi-Fi/VPN), renewed, and revoked correctly — proven on a named pilot before it scales.

The GCC High differences that matter (doc-confirmed)

Drawn from Microsoft's *Intune Government Service overview* and the *Certificate Connector* / *Cloud PKI* docs:

- **Microsoft Cloud PKI is listed under "Intune features planned for GCC High and DoD"** — i.e. **currently not available and not supported** — annotated **"(GCC High only)"**, meaning the plan covers GCC High but **not DoD**. **Do not design on Cloud PKI in GCC High today.**
- **SCEP, PKCS, imported PKCS (PFX), and trusted certificate profiles are available** in GCC High as standard MDM device-configuration features.
- **The Certificate Connector for Microsoft Intune is available** (installed from `intune.microsoft.us`); **NDES is required** for SCEP against a Microsoft (AD CS) CA. **Partner-CA connectors work only where the vendor supports GCC High** — verify.
- **Scripts must target US Gov Graph** (`Connect-MgGraph -Environment USGov`, `graph.microsoft.us`).

Where a GCC High specific could not be confirmed (partner-CA support, derived-credential issuers), the solution states the position and **explicitly flags "verify for GCC High"** rather than inventing menus or parity claims.

Business outcome

A fragile or manual certificate estate — hand-issued certificates, no renewal story, no mobile visibility — becomes a **managed** one on a US sovereign cloud: certificates **issued, trusted, renewed, and revoked** through Intune on the **supported** path, validated on a pilot, and safe to expand — with a **clean forward road to Cloud PKI** the moment Microsoft ships it to GCC High. Self-guided and tested, at a fraction of consulting cost.

Deployment model: Assess → Configure → Pilot → Validate → Expand.

Solution Overview — Cloud PKI Integration with Intune (Government / GCC High)

Read this first — Cloud PKI availability in GCC High

Microsoft Cloud PKI is *not yet available* in GCC High, and is *not planned* for DoD. In Microsoft's *Intune Government Service overview*, Cloud PKI is listed under **"Intune features planned for GCC High and DoD"** — the table of features that are **currently not available and not supported** but where planning has started — annotated **"(GCC High only)."** That annotation means the *plan* to deliver Cloud PKI covers **GCC High**, not **DoD**.

So on a US sovereign-cloud tenant today:

- **You cannot deploy Microsoft Cloud PKI in GCC High yet** (it is planned; Microsoft public-sector messaging points at a Government-cloud arrival during 2026 — **treat as an ETA, verify before you plan a go-live**).
- **DoD is not on the Cloud PKI roadmap at all.**
- **The supported way to issue certificates to Intune-managed devices in GCC High is SCEP and/or PKCS certificate profiles delivered by the Intune Certificate Connector**, **anchored to your own** AD CS **(or a partner CA that supports the government cloud). This edition builds the certificate program on that** supported **foundation, and treats Cloud PKI as a** future migration** you slot in once Microsoft ships it to GCC High.

This is the difference that makes this a **different solution, not a re-login**: a Commercial "stand up Cloud PKI in the Intune admin center" runbook **does not apply in GCC High** — the capability is simply not there.

> ***Source:*** *Microsoft Learn — *Microsoft Intune Government Service overview*, "Intune features planned for GCC High and DoD," row* ***Cloud PKI (GCC High only)*** *(`https://learn.microsoft.com/en-us/intune/fundamentals/government-service`, doc dated 2026-06-04). Full citations in* `gcc-high-parity-notes.md`.

What this solution delivers

This is the **Government (GCC High)** edition of the certificate-management modernization solution. It walks a competent-but-novice administrator through issuing and managing device/user certificates on a **US sovereign-cloud (GCC High)** tenant, the repeatable way:

1. **A trusted-root/intermediate trust chain** published to Intune-managed devices via **trusted certificate profiles**.

1. **SCEP certificate profiles** (dynamic, per-device enrollment via NDES + the Intune Certificate

Connector) and/or **PKCS certificate profiles** (connector-issued from AD CS) for Wi-Fi, VPN, Wi-Fi/802.1X, S/MIME, and device-identity scenarios.

1. **Certificate lifecycle hygiene** — issuance, renewal, and revocation — validated on a pilot,

with drift-detection scripts, **before** you scale.

Together these turn a **fragile or manual certificate estate** (hand-issued certs, no renewal story, GPO-only distribution, no visibility on mobile) into a **managed** one: certificates **issued, trusted, renewed, and revoked** through Intune — proven on a pilot before you scale — on the **supported** government path, with a clean forward road to Cloud PKI when it arrives.

Why GCC High is a different solution, not a re-login

Certificate management in **GCC High is not the Commercial portal with a different sign-in.** Two things differ in ways that break Commercial runbooks:

• **The portals and endpoints are different sovereign-cloud hosts.** You administer Intune at

`intune.microsoft.us`, sign in at `login.microsoftonline.us`, and your scripts must target **US Gov Graph endpoints** (`Connect-MgGraph -Environment USGov`, `graph.microsoft.us`). Point a Commercial runbook at `intune.microsoft.com` and you are in the wrong cloud. (Intune GCC High/DoD runs on **Azure Government**, a physically separate **IL4/IL5** datacenter.)

• **Feature parity lags Commercial — and for this topic the gap is decisive.** The marquee Commercial

capability for this solution, **Microsoft Cloud PKI, is not available in GCC High** (planned) and **not planned for DoD**. Following a Commercial "enable Cloud PKI" runbook silently fails: the option is not there. The **supported** government approach — SCEP/PKCS via the Intune Certificate Connector — is a different design with different prerequisites (an on-prem CA, NDES, a connector).

This edition calls out the correct GCC High portals/endpoints at each step, and flags the parity gap **where it matters**, so a government tenant is configured correctly. The full parity reference lives in `gcc-high-parity-notes.md` and is cited throughout.

The certificate model (how the pieces fit)

• A **trusted certificate profile** deploys your **root** (and any **intermediate**) CA public

certificate to devices so they *trust* certificates issued from that chain.

• A **SCEP certificate profile** tells the device to generate a key pair on-device (**the private

key never leaves the device**) and request a leaf certificate. With an** AD CS **CA, the request is serviced by** NDES**, validated by the** Intune Certificate Connector**, and signed by your CA.

• A **PKCS certificate profile** has the **Intune Certificate Connector** request the certificate

from **AD CS** on the device's behalf and deliver it. **Imported PKCS (PFX)** covers per-user S/MIME.

• **Renewal and revocation** are driven from Intune (renewal thresholds on the profile; revocation

on unenroll/wipe or manually), with the connector performing revocation against the CA.

The design principle is **assign once, repeat everywhere**: one well-named trust profile plus a small set of named SCEP/PKCS profiles, assigned to a pilot group and then expanded — instead of one-off, hand-issued certificates.

> ***Where Cloud PKI fits later:*** *Cloud PKI replaces the on-prem CA + NDES + connector with a cloud-hosted root/issuing CA and a built-in SCEP registration authority. When it reaches GCC High, the **trusted-root and SCEP profiles** you build here map onto a Cloud PKI *issuing CA* with minimal rework — which is why building the profile layer correctly now pays off.*
> *(`gcc-high-parity-notes.md` §2.)*

Commercial vs GCC High — what's different

*(Doc-confirmed from Microsoft's *Intune Government Service overview* and *Microsoft Cloud PKI* / *Certificate Connector* docs unless flagged. See `gcc-high-parity-notes.md` for the full table and citations.)*

Portals & endpoints (always different):

Purpose	Commercial	GCC High
Sign-in	`login.microsoftonline.com`	`login.microsoftonline.us`
Microsoft 365 admin	`admin.microsoft.com`	`portal.office365.us`
Intune admin center	`intune.microsoft.com` / `endpoint.microsoft.com`	`intune.microsoft.us`
Security portal	`security.microsoft.com`	`security.microsoft.us`
Entra admin	`entra.microsoft.com`	`entra.microsoft.us`
Azure portal	`portal.azure.com`	`portal.azure.us`
Graph (scripts)	`graph.microsoft.com` `(-Environment Global)`	`graph.microsoft.us` `(-Environment USGov)`

The parity gap that changes the whole design (GCC High status in parentheses):

- **Microsoft Cloud PKI** is **not available** in GCC High (*planned, GCC High only*) and **not planned for DoD**. Do not design on it. **Use** SCEP/PKCS + the Intune Certificate Connector **(below), which is** supported.
- **SCEP certificate profiles, PKCS certificate profiles, imported PKCS (PFX), and trusted certificate profiles **are** available **in GCC High as** standard MDM / device-configuration** features.
- **The Certificate Connector for Microsoft Intune** is **available**; install it from `intune.microsoft.us`. Its network requirements are the **same as a managed device** — in GCC High that means the **US Gov** endpoints, plus `autoupdate.msappproxy.net:443` for auto-update.
- **NDES** is **required** to issue SCEP from an **AD CS (Microsoft) CA**. (Third-party SCEP CAs do not require the connector.)
- **Partner-CA connectors (e.g. DigiCert)** work **only where the partner supports the GCC High

government cloud **—** verify for GCC High** before designing on one (mirrors how MTD connectors are limited to vendors that support GCC High).

- **Remote Help** is also **planned/not-yet-available**; **classic Windows Autopilot** is **not available** in GCC High — noted because certificate rollouts often ride enrollment flows.

Where a specific menu, connector, or Cloud PKI option is not visible in `intune.microsoft.us`, treat it as a **GCC High parity gap, not a permissions problem** — re-check the parity table before troubleshooting.

The standard deployment model

Every step in this solution follows the same repeatable model, with an explicit **pass/fail gate** between phases:

Assess → **Configure** → **Pilot** → **Validate** → **Expand.**

- **Assess** — confirm licensing (Gov G3/G5), roles, portal access, an available CA + NDES + connector, and (critically) **that Cloud PKI is not being assumed** (the parity check).
- **Configure** — build the trusted-root profile and the SCEP/PKCS profiles. Name every object verbatim.
- **Pilot** — assign to a **named pilot group** only.
- **Validate** — run the pass/fail gates in `05-testing.md` against the pilot (certs issued, trusted, Wi-Fi/VPN authenticate, renewal/revocation behave).
- **Expand** — widen scope group-by-group only after the gate passes.

What's in the box

- This overview, a **Requirements** guide, an ordered **Implementation** guide, **Testing** (pass/fail gates), **Troubleshooting** (with GCC-High-specific gotchas), **Best Practices**, and a **Deliverables Summary** + **Executive Summary**.
- A **GCC High parity notes** reference (`gcc-high-parity-notes.md`) with per-feature status and Microsoft citations — including the Cloud-PKI-not-available finding.
- **Five PowerShell scripts** (Graph SDK, US Gov endpoints): Discovery, Validation, Report, Remediation, and Set-ScopeTargeting — read-only tools default to read-only; change tools default to `-WhatIf`.

> ***Sources:*** *Microsoft Learn — *Microsoft Intune Government Service overview* (`.../fundamentals/government-service`); *Microsoft Cloud PKI for Microsoft Intune* (`.../cloud-pki/`); *Overview of Certificate Connector for Microsoft Intune* (`.../protect/certificate-connector-overview`); MSPortals.io US Government portals. Full citations in* `gcc-high-parity-notes.md`.

Requirements — Cloud PKI Integration with Intune (Government / GCC High)

Confirm every prerequisite below **before** you configure anything. This edition assumes a **GCC High** tenant throughout — the portals and endpoints are the US sovereign-cloud hosts, not the Commercial ones. Category-level prerequisites (Entra tenant, admin roles, pilot group, network access, change control) are in `global-requirements-security.md`; the items here are **certificate- and GCC-High-specific.**

Each requirement is written as **Description · Why · Verify · Implement if missing.**

> ***Before anything else — the Cloud PKI reality check (R0).*** *Because **Microsoft Cloud PKI is not available in GCC High (planned) and not planned for DoD**, this requirements list is built for the **supported** path: **SCEP/PKCS certificate profiles via the Intune Certificate Connector**, anchored to your own CA. If your plan assumed cloud-hosted Cloud PKI issuing CAs, **stop and re-scope** to the connector model here. (`gcc-high-parity-notes.md` §2.)*

R0. Confirm Cloud PKI is out of scope for now (parity acknowledgement)

Description: A recorded decision that this deployment uses **SCEP/PKCS + the Intune Certificate Connector**, because **Cloud PKI is not yet available in GCC High** (and not planned for DoD). **Why:** Building on Cloud PKI in GCC High wastes the whole project — the capability is not present. **Verify:** In `gcc-high-parity-notes.md` §2, and against the live *Intune Government Service overview*, confirm **Cloud PKI is still under "features planned"** for GCC High (i.e. not yet GA). If it has since shipped to GCC High, you may add a Cloud PKI issuing CA — otherwise use the connector. **Implement if missing:** Record the scope decision in the design (connector-based today; Cloud PKI as a future migration once Microsoft ships it to GCC High).

R1. A Microsoft 365 Government – GCC High tenant

Description: An active **Microsoft 365 Government – GCC High** tenant (US sovereign cloud), not a Commercial or GCC (moderate) tenant. (For Intune, **GCC = Commercial**; only **GCC High/DoD** run in the government datacenter.) **Why:** GCC High uses different sovereign endpoints and a different Intune feature-parity profile. Commercial steps and URLs will not work, and Cloud PKI you may plan for does not exist here. **Verify:** Sign in at `https://login.microsoftonline.us` and open `https://intune.microsoft.us`. The Intune admin center loads at the `.us` host and branding indicates a US Government (GCC High) tenant. **Implement if missing:** GCC High requires eligibility validation and onboarding through Microsoft / an authorized Gov reseller — you cannot self-provision it like Commercial. This is a procurement action, not a portal click.

R2. Government Intune licensing (G3 / G5 with Intune Plan 1)

Description: Intune-bearing **Government** licensing — **Microsoft 365 Government G3/G5** (which include **Intune Plan 1**) or standalone Intune Plan 1 for Gov. **Why: SCEP/PKCS certificate profiles and the Certificate Connector are standard Intune Plan 1 functionality — no add-on required.** (Cloud PKI, when it reaches GCC High, would be an **Intune Suite / standalone add-on** on top — but it is not in scope today.) **Verify:** `portal.office365.us` → **Billing** → **Licenses**; confirm **Microsoft 365 G3/G5** or Intune Plan 1 (Gov), assigned to the admin and pilot users. **Implement if missing:** Acquire/assign the

appropriate Gov SKU through your Gov licensing channel. (See `gcc-high-parity-notes.md` §5.)

R3. Intune administrative roles

Description: Intune Administrator (for certificate profiles and connectors), plus the ability to create Entra groups. For least privilege, use the custom-role certificate permissions rather than Global Administrator. **Why:** Creating trusted-certificate, SCEP, and PKCS profiles and installing the connector requires elevated Intune rights. **Verify:** `entra.microsoft.us` → **Roles and administrators**; confirm **Intune Administrator** on the operator. **Implement if missing:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes only.

R4. An issuing Certification Authority (AD CS) reachable on-network

Description: An enterprise **Active Directory Certificate Services (AD CS)** CA (root + issuing/intermediate) whose issuing CA can sign device/user certificates, reachable from the server that will host NDES/the connector. **Why:** In GCC High (no Cloud PKI), the leaf certificates are signed by **your** CA. SCEP with a Microsoft CA requires AD CS; PKCS is issued from AD CS by the connector. **Verify:** On the CA, `certutil -ping` / the **Certification Authority** console shows the issuing CA online; the intended **certificate templates** exist (e.g. a Wi-Fi/VPN client-auth template) and the NDES/connector service account can enroll them. **Implement if missing:** Stand up / designate an AD CS issuing CA, create the client-auth certificate template(s), and grant the service account **Enroll** rights. (If you use a **partner CA** instead of AD CS, see R7 and **verify the partner supports GCC High**.)

R5. NDES (for SCEP with a Microsoft CA)

Description: The **Network Device Enrollment Service (NDES)** role installed and published (for SCEP against an AD CS CA), on a supported Windows Server, reachable by managed devices (typically via a reverse proxy such as Azure AD Application Proxy alternative / Web Application Proxy in the Gov environment). **Why:** SCEP leaf issuance from a Microsoft CA is serviced by NDES; the Intune Certificate Connector validates the SCEP request and NDES forwards it to the CA. **Verify:** The NDES `mscep`/`mscep_admin` endpoints respond on the NDES server; the NDES service account and IIS are healthy. **Implement if missing:** Install the **AD CS** → **Network Device Enrollment Service** role, configure the NDES service account and templates, and publish the SCEP URL to devices through your Gov reverse proxy. (Skip NDES if you are doing **PKCS only**, or using a **third-party SCEP CA** that does not require the connector.)

R6. The Certificate Connector for Microsoft Intune (installed from the GCC High portal)

Description: The **Certificate Connector for Microsoft Intune** installed on a supported Windows Server (SCEP and/or PKCS and revocation features), downloaded from `intune.microsoft.us`. **Why:** The connector is the bridge between Intune (US Gov cloud) and your on-prem CA/NDES — it processes SCEP validation, PKCS issuance, imported PFX, and revocation. **Verify:** `intune.microsoft.us` → **Tenant administration** → **Connectors and tokens** → **Certificate connectors**; the connector shows **Active/Healthy** (not Warning/Error). Connector **Event logs** (Applications and Services → Microsoft → Intune → Certificate Connectors) show success events. **Implement if missing:** From `intune.microsoft.us`, download the connector, install the required features, assign the service

account, and allow its outbound HTTPS to the **US Gov** endpoints plus `autoupdate.msappproxy.net:443`. Keep it on a supported version (six-month support lifecycle). Source: *Certificate Connector overview*.

R7. (If using a partner CA) partner-CA support for GCC High

Description: If you issue via a **non-Microsoft CA** (e.g. DigiCert) through the connector, confirmation that the **partner CA integration is supported in the GCC High government cloud**. **Why:** Like MTD connectors, partner integrations are only usable where the vendor supports GCC High. A Commercial-only partner integration will not function in the sovereign cloud. **Verify:** Confirm with the partner and in `intune.microsoft.us` that the partner-CA connector is available for your GCC High tenant. **Flagged: verify for GCC High. Implement if missing:** Either select a partner that supports GCC High, or use **AD CS** (R4/R5).

R8. Network / filtering allow-list for GCC High endpoints

Description: Outbound HTTPS to the **GCC High sovereign endpoints** for the Intune portal, device management, and the connector — plus device reachability to the SCEP/NDES URL. **Why:** Enrollment, profile sync, certificate issuance/validation, connector auto-update, and script access all use the `.us` Gov hosts. If only the `.com` Commercial hosts are allowed, GCC High certificate flows will not function. **Verify:** From an admin workstation and from a managed device, reach `https://login.microsoftonline.us`, `https://intune.microsoft.us`, `https://graph.microsoft.us`, and (for auto-update) `autoupdate.msappproxy.net:443`; from a device, reach your published **NDES/SCEP** URL. **Implement if missing:** Allow Microsoft's published **Office 365 U.S. Government GCC High** URL/IP ranges (the Gov endpoint list is separate from the Commercial "Worldwide" list), plus your NDES reverse-proxy URL, through proxy/firewall.

R9. Enrolled devices on a supported OS, and a pilot scope + change control

Description: Target devices **enrolled in Intune** on a supported OS (Windows/iOS/iPadOS/ macOS/Android), a named **pilot group**, and a **change-control/rollback** process (certificate, Wi-Fi, and VPN changes can lock users out). **Why:** Certificate profiles only reach enrolled devices; certificate/auth changes are high-impact and must be piloted with a rollback. SCEP/PKCS profiles support the same platforms Intune supports in GCC High. **Verify:** `intune.microsoft.us` → **Devices** → **All devices** (targets present/compliant); a pilot Entra security group named `SG-CPKI-Pilot` exists (`entra.microsoft.us` → Groups); a change ticket template with approver, window, and rollback exists. **Implement if missing:** Enroll devices; create `SG-CPKI-Pilot` in `entra.microsoft.us`; adopt the change-control process from `global-requirements-security.md` (G8).

R10. Parity confirmation for the certificate features you intend to use

Description: A completed check of the **GCC High parity table** for every capability this deployment depends on — first and foremost **Cloud PKI availability**, then partner-CA support and any derived-credential issuer. **Why:** Cloud PKI is **not available** in GCC High; a partner CA may be Commercial-only; derived credentials depend on issuer support. Designing around a missing feature wastes the pilot. **Verify:** Work through `gcc-high-parity-notes.md`; for each capability confirm status against the live Microsoft doc **and** that the option/connector is visible in `intune.microsoft.us`. **Implement if missing:** Where a feature is *planned / not available*, use the GA alternative (SCEP/ PKCS

via AD CS + connector), and record the substitution in the design.

The deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Implementation — Cloud PKI Integration with Intune (Government / GCC High)

Follow these steps **in order**. Every navigation path uses the **GCC High** portals — sign in at `https://login.microsoftonline.us` and work in `https://intune.microsoft.us` (the Microsoft Intune admin center for the US government cloud). If a portal loads at a `.com` host, stop — you are in the Commercial cloud.

> ***Cloud PKI is not available in GCC High (planned) and not planned for DoD.*** *This implementation therefore builds the* ***supported*** *path —* ***trusted-root + SCEP/PKCS certificate profiles via the Intune Certificate Connector****, anchored to your* ***AD CS*** *CA. If Cloud PKI has since shipped to your GCC High tenant (verify in* `intune.microsoft.us` *and* `gcc-high-parity-notes.md` *§2), you may substitute a Cloud PKI *issuing CA* for the AD CS/NDES/connector back-end — the* ***profile*** *steps below (trusted root + SCEP) still apply.*

Naming — use these object names verbatim (reuse them exactly in later steps and in the scripts):

Object	Name to create and reuse
Pilot Entra security group	`SG-CPKI-Pilot`
Trusted certificate profile (root/intermediate)	`Trust-Root-CA`
SCEP certificate profile (Wi-Fi/VPN client auth)	`SCEP-Device-ClientAuth`
PKCS certificate profile (alternative / user certs)	`PKCS-User-ClientAuth`
Certificate Connector (server name reference)	`CertConnector-01`

GCC High parity reminders are inlined at the step where they matter. Full detail: `gcc-high-parity-notes.md`.

Phase 0 — Assess (gate before you build)

1. Sign in at `https://login.microsoftonline.us`; open `https://intune.microsoft.us`.

Confirm **Devices** → **Configuration** and **Tenant administration** → **Connectors and tokens** → **Certificate connectors** are present.

1. **Confirm Cloud PKI is out of scope (R0).** In `intune.microsoft.us`, confirm there is **no**

supported Cloud PKI CA creation blade for your tenant (Cloud PKI is planned, not GA in GCC High). Record the connector-based scope decision. (`gcc-high-parity-notes.md` §2.)

1. Confirm licensing (R2) at `https://portal.office365.us` → **Billing** → **Licenses** — G3/G5 / Intune

Plan 1 present.

1. Confirm roles (R3) at `entra.microsoft.us` → **Roles and administrators** — **Intune

Administrator**.

1. Confirm the **CA + NDES + connector** back-end (R4–R6): AD CS issuing CA online, NDES published,

and the **Certificate Connector** shows **Active/Healthy** under **Tenant administration** → **Connectors and tokens** → **Certificate connectors**.

1. Confirm `SG-CPKI-Pilot` exists at `entra.microsoft.us` → **Groups**. If not: **Groups → New

group → Security**, name it `SG-CPKI-Pilot`, add pilot users/devices.

1. **Run the parity check (R10)** for partner-CA / derived-credential dependencies.

Gate: portal loads at `.us`; Cloud PKI confirmed out-of-scope; licensing/roles present; CA + NDES + connector healthy; pilot group exists. Do not proceed until all pass.

Phase 1 — Publish the trust chain (trusted certificate profile)

Step 1.1 — Export the CA chain

1. From your **AD CS** issuing CA, export the **root** (and any **intermediate**) CA certificate as

DER (.cer) — public certificate only, **no private key**.

Step 1.2 — Create the trusted certificate profile

1. In `intune.microsoft.us` go to **Devices** → **Configuration** → **+ Create** → **New policy**.
2. Choose the target **platform** (create one per platform you manage — Windows, iOS/iPadOS, macOS,

Android) and **Profile type = Templates** → **Trusted certificate**.

1. Upload the **root** `.cer`. For an intermediate, create a **second** trusted certificate profile

with the intermediate `.cer` (and, on Windows, set the destination store appropriately).

1. **Name the profile** `Trust-Root-CA` (append the platform, e.g. `Trust-Root-CA-Windows`).
2. **Assignments:** assign to `SG-CPKI-Pilot` only.

Gate: on a pilot device, the root/intermediate appears in the device's trusted store after sync. If not, see `06-troubleshooting.md`.

Phase 2 — SCEP certificate profile (dynamic per-device issuance)

> ***GCC High note:*** *SCEP profiles are a* ***standard MDM device-configuration*** *feature and* ***are available*** *in GCC High. Leaf certificates are signed by* ***your AD CS CA*** *(Cloud PKI is not available), so SCEP here* ***requires NDES + the Certificate Connector*** *(R5/R6). The SCEP URL you enter must be your* ***published Gov NDES URL****. (`gcc-high-parity-notes.md` §3.)*

Step 2.1 — Confirm the SCEP back-end is healthy

1. `intune.microsoft.us` → **Tenant administration → Connectors and tokens → Certificate

connectors **—** `CertConnector-01` **is** Active **with the** SCEP** feature enabled.

1. Your **NDES** endpoint is reachable from a managed device (test the published SCEP URL).

Step 2.2 — Create the SCEP profile

1. **Devices** → **Configuration** → **+ Create** → **New policy** → platform → **Templates** → **SCEP certificate**.

2. Configure:

• **Certificate type:** Device (or User, per scenario).

• **Subject name format / SAN:** per your template (e.g. `CN={{DeviceName}}`, SAN = `{{AAD_Device_ID}}` or UPN for user certs).

• **Certificate validity period / key storage / key usage / EKU:** match the **AD CS template** (e.g. **Client Authentication** EKU for Wi-Fi/VPN).

• **Root certificate:** select the `Trust-Root-CA` trusted profile from Phase 1.

• **SCEP Server URLs:** your **published Gov NDES SCEP URL**.

1. **Keep certificate extensions standard.** The SCEP validation service **blocks unknown extension OIDs** (connector v6.2510.3.2002+); only the allow-listed extensions (Basic Constraints, Subject Key Identifier, CA Version, Certificate Template Name, etc.) are honored — do not add custom OIDs. (`gcc-high-parity-notes.md` §3.)

1. **Name the profile** `SCEP-Device-ClientAuth`; **assign to** `SG-CPKI-Pilot` only.

Gate: a pilot device receives and installs a SCEP-issued certificate (visible in the device cert store / Intune device certificate report). Connector **Event ID 4006** = SCEP issued success.

Phase 3 — PKCS certificate profile (alternative / user certificates)

> ***GCC High note:*** *PKCS profiles and imported PKCS (PFX) are **available** in GCC High (standard MDM). PKCS certificates are requested from **AD CS by the connector** (no NDES needed for PKCS).*

Step 3.1 — Confirm the connector PKCS feature

1. `intune.microsoft.us` → **Certificate connectors** — `CertConnector-01` has the **PKCS** feature enabled and the service account can enroll the AD CS template.

Step 3.2 — Create the PKCS profile

1. **Devices** → **Configuration** → **+ Create** → **New policy** → platform → **Templates** → **PKCS certificate**.

2. Configure **Certification authority** (the AD CS CA FQDN), **CA name**, **Certificate template name **(exact AD CS template),** Subject/SAN**,** key usage/EKU**, and select `Trust-Root-CA` as the root.

1. **Name the profile** `PKCS-User-ClientAuth`; **assign to** `SG-CPKI-Pilot` only.

Gate: a pilot user/device receives the PKCS certificate; connector **Event ID 1050** = PKCS issued success. (For **S/MIME**, use **imported PKCS/PFX** per-user instead.)

Phase 4 — Consume the certificate (Wi-Fi / VPN) and prove renewal/revocation

Step 4.1 — Bind the certificate to a Wi-Fi or VPN profile

1. **Devices** → **Configuration** → **+ Create** → **New policy** → platform → **Templates** → **Wi-Fi** (or **VPN**).
1. For **802.1X / EAP-TLS Wi-Fi** or **certificate-based VPN**, set authentication to use a **client certificate** and select `SCEP-Device-ClientAuth` (or `PKCS-User-ClientAuth`) as the client certificate source, and `Trust-Root-CA` as the trusted root.
1. **Name and assign to** `SG-CPKI-Pilot`**.**

Gate: a pilot device authenticates to the certificate-protected Wi-Fi/VPN.

Step 4.2 — Renewal and revocation

1. **Renewal:** confirm the SCEP/PKCS profile's **renewal threshold** (e.g. renew at 20% remaining) so certificates auto-renew before expiry.
1. **Revocation:** revoke a test certificate (retire/wipe the pilot device, or revoke on the CA) and confirm the connector processes the revocation (**Event ID 3050** = revoke success) and the certificate is no longer trusted.

Gate: renewal fires before expiry; revocation removes trust. Both go through change control (G8).

Phase 5 — Pilot → Validate → Expand

1. **Pilot** is already scoped to `SG-CPKI-Pilot` across the trust profile and SCEP/PKCS profiles.
2. **Validate** using the pass/fail gates in `05-testing.md`. Run `CPKIG-Validation.ps1` and `CPKIG-Report.ps1` (US Gov Graph) to confirm profile presence, assignment, and issued-cert coverage.
1. **Expand** only after gates pass: widen assignments group-by-group using `CPKIG-Set-ScopeTargeting.ps1` (defaults to `-WhatIf`). Do not assign certificate/Wi-Fi/VPN profiles tenant-wide on day one.

> ***Future migration to Cloud PKI (when it reaches GCC High):*** *re-check the parity table. If Cloud PKI becomes GA in your GCC High tenant, create a Cloud PKI* ***root/issuing CA****, re-point the* ***trusted-root*** *and* ***SCEP*** *profiles at the Cloud PKI issuing CA / its SCEP registration authority URL, validate on the pilot, then retire NDES + the connector. Plan it as its own Assess→…→Expand cycle. (*`gcc-high-parity-notes.md` *§2.)*

The deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

> ***Sources:*** *Microsoft Learn — *Microsoft Intune Government Service overview* (Cloud PKI planned; standard MDM available); *Overview of Certificate Connector for Microsoft Intune* (SCEP/PKCS,*

*NDES, OID allow-list, auto-update); *Microsoft Cloud PKI for Microsoft Intune* (future model). Portals:* `intune.microsoft.us`, `portal.office365.us`, `entra.microsoft.us`, `login.microsoftonline.us`. *Full citations in* `gcc-high-parity-notes.md`.

Testing — Cloud PKI Integration with Intune (Government / GCC High)

Run these gates against the **pilot** (`SG-CPKI-Pilot`) before expanding. Each gate has an explicit **PASS / FAIL** condition. A FAIL sends you to `06-troubleshooting.md`; do not expand scope until every applicable gate is **PASS**. Portals are the GCC High `.us` hosts; scripts connect with `Connect-MgGraph -Environment USGov`.

Gate 0 — Sovereign-cloud sanity (must pass first)

- **Test:** In a browser, load `https://intune.microsoft.us`; run `CPKIG-Discovery.ps1` and confirm

it connected to the **US Gov** Graph endpoint.

- **PASS:** Portal loads at a `.us` host; script reports environment `USGov` / `graph.microsoft.us`.
- **FAIL:** Any portal or script resolves to a `.com` host → you are targeting Commercial; stop and

fix endpoint/environment before any other test.

Gate 1 — Cloud PKI parity acknowledged (not assumed)

- **Test:** Confirm the design uses **SCEP/PKCS via the connector**, not Cloud PKI; and re-confirm in

`gcc-high-parity-notes.md` §2 and the live *Intune Government Service overview* that **Cloud PKI is still "planned" (not GA)** in GCC High.

- **PASS:** Connector-based design recorded; Cloud PKI confirmed not-yet-available (or, if it has

shipped, the design explicitly opts into it). **FAIL:** Design assumes a Cloud PKI issuing CA that does not exist in the tenant → re-scope to the connector model.

Gate 2 — Licensing & roles present

- **Test:** `portal.office365.us` → `Billing` → `Licenses` shows G3/G5 / Intune Plan 1;

`entra.microsoft.us` → `Roles and administrators` shows **Intune Administrator**.

- **PASS:** Both present. **FAIL:** Missing SKU or role → cannot author profiles / install connector.

Gate 3 — CA + NDES + connector healthy

- **Test:** `intune.microsoft.us → Tenant administration → Connectors and tokens → Certificate

connectors shows CertConnector-01` **Active/Healthy**; the AD CS issuing CA is online; the published NDES/SCEP URL responds from a device.

- **PASS:** Connector Active (not Warning/Error); CA online; SCEP URL reachable. **FAIL:** Connector Warning/Error (deprecated/out-of-support version), CA offline, or SCEP URL unreachable → fix before issuing.

Gate 4 — Trust chain deployed

- **Test:** On a pilot device (after sync), inspect the trusted certificate store.
- **PASS:** The `Trust-Root-CA` root (and intermediate, if used) is present in the correct store.
- **FAIL:** Root missing → verify the trusted certificate profile platform/store and its assignment to `SG-CPKI-Pilot`.

Gate 5 — SCEP certificate issued

- **Test:** A pilot device receives `SCEP-Device-ClientAuth` and requests a certificate.
- **PASS:** The leaf certificate installs on the device; Intune device certificate report shows it; connector **Event ID 4006** (SCEP issued success). Private key is on-device (never left it).
- **FAIL:** No certificate → check NDES/SCEP URL, connector SCEP feature, template EKU/permissions, and that no unknown extension OID tripped the SCEP validation allow-list.

Gate 6 — PKCS certificate issued (if in scope)

- **Test:** A pilot user/device receives `PKCS-User-ClientAuth` (or imported PFX for S/MIME).
- **PASS:** Certificate installs; connector **Event ID 1050** (PKCS issued success).
- **FAIL:** No certificate → verify the connector PKCS feature, CA name/template name exactness, and service-account enroll rights.

Gate 7 — Certificate consumed (Wi-Fi / VPN authenticates)

- **Test:** A pilot device connects to the certificate-protected **Wi-Fi (EAP-TLS)** or **VPN**.
- **PASS:** The device authenticates using the issued client certificate against `Trust-Root-CA`.
- **FAIL:** Auth fails → confirm the Wi-Fi/VPN profile references the correct client-cert source and trusted root, and the certificate's EKU is Client Authentication.

Gate 8 — Renewal works

- **Test:** Confirm the SCEP/PKCS profile renewal threshold; simulate/verify renewal behavior before expiry (or shorten validity on a test template to observe renewal).
- **PASS:** A new certificate is issued before expiry without user action; old one is superseded.
- **FAIL:** No renewal → check the renewal-threshold setting and connector/CA health.

Gate 9 — Revocation works

- **Test:** Retire/wipe the pilot device (or revoke on the CA) and re-test access.
- **PASS:** The connector processes revocation (**Event ID 3050**); the certificate is no longer

trusted and access is denied.

- **FAIL:** Certificate still valid → verify revocation reached the CA/CRL and the relying party uses

the CRL/AIA.

Gate 10 — Script validation clean

- **Test:** Run `CPKIG-Validation.ps1` and `CPKIG-Report.ps1` (US Gov Graph).
- **PASS:** Validation reports `Trust-Root-CA`, `SCEP-Device-ClientAuth` (and `PKCS-User-ClientAuth`

if in scope) present and assigned to `SG-CPKI-Pilot`; the connector is healthy; report lists issued certificates with no unexpected gaps.

- **FAIL:** Reported gaps → remediate (see `CPKIG-Remediation.ps1`, which defaults to `-WhatIf`) and

re-run.

Expansion gate: proceed from Pilot to Expand **only** when Gates 0–5 and 7 are PASS (plus 6/8/9 where in scope), the connector is healthy, and any certificate/Wi-Fi/VPN assignment change has change-control approval.

The deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Troubleshooting — Cloud PKI Integration with Intune (Government / GCC High)

Work top-down. The first section covers the failures that are **unique to GCC High** — check these before generic certificate troubleshooting, because in GCC High "it's not there" is often a **parity gap** (above all, **Cloud PKI**), not a misconfiguration.

A. GCC-High-specific gotchas

A1. "Where is Cloud PKI? The blade/option isn't there."

Symptom: A Commercial guide says to create a **Cloud PKI** root/issuing CA in the Intune admin center, but the option does not exist in `intune.microsoft.us`. **Likely cause: Cloud PKI is not available in GCC High** — it is listed under *"Intune features planned for GCC High and DoD"* (not yet GA), and is **not planned for DoD** at all. This is a **parity gap, not a bug. Check / fix:** Confirm in `gcc-high-parity-notes.md` §2 and the live *Intune Government Service overview*. Use the **supported** path instead: **trusted-root + SCEP/PKCS certificate profiles via the Intune Certificate**

Connector (see `03-implementation.md`). **Do not file a support case for a planned feature.** Re-check the roadmap periodically — Cloud PKI is planned for GCC High and may become GA.

A2. Wrong cloud — portal or script hit a `.com` host

Symptom: Sign-in loops, "tenant not found", empty portal, or a script authenticates but sees no Intune objects. **Likely cause:** You reached the **Commercial** cloud (`intune.microsoft.com`, `login.microsoftonline.com`, or `Connect-MgGraph -Environment Global`) instead of GCC High. **Fix:** Use the `.us` hosts only — `login.microsoftonline.us`, `intune.microsoft.us`, `security.microsoft.us`, `entra.microsoft.us`, `portal.office365.us`. In scripts, connect with `Connect-MgGraph -Environment USGov` (DoD: `USGovDoD`). Verify with `(Get-MgContext).Environment` = `USGov`. Run `CPKIG-Discovery.ps1`, which reports the environment.

A3. Endpoint blocked by filtering

Symptom: Portal will not load, the connector cannot reach the service, or devices cannot reach the SCEP URL — on-network only. **Likely cause:** Proxy/firewall allows the **Commercial** Office 365 endpoint list but not the **Office 365 U.S. Government GCC High** list (the `.us` sovereign hosts are a separate published set), or the NDES reverse-proxy URL is blocked. **Fix:** Allow the GCC High URL/IP ranges (portals, `graph.microsoft.us`, `login.microsoftonline.us`, the connector's endpoints and `autoupdate.msappproxy.net:443`), plus your published NDES/SCEP URL. Re-test R8 reachability from an on-network workstation **and** a device.

A4. Partner-CA connector unavailable in GCC High

Symptom: A non-Microsoft CA integration (e.g. DigiCert) you used in Commercial is missing or non-functional in the GCC High tenant. **Likely cause:** The partner integration is **Commercial-only** or not yet supported in the government cloud (mirrors how MTD connectors are limited to vendors that support GCC High). **Fix: Verify for GCC High** with the partner and in `intune.microsoft.us`. If unsupported, use **AD CS** (R4/R5) instead. Record the decision.

B. Certificate Connector not healthy

- **Cause:** Connector service down, out-of-support version (**Warning**→**Error**), service-account problem, or blocked outbound/`autoupdate.msappproxy.net`.
- **Fix:** `intune.microsoft.us → Tenant administration → Connectors and tokens → Certificate connectors — check status. Update the connector (six-month support lifecycle; out-of-support connectors stop issuing/revoking). Confirm the service account, and the **Event logs** (Applications and Services → Microsoft → Intune → Certificate Connectors, event IDs 5xxx = health). Ensure auto-update reaches autoupdate.msappproxy.net:443`.

C. Trust chain not on the device

- **Cause:** `Trust-Root-CA` profile targets the wrong platform/store, or is not assigned to `SG-CPKI-Pilot`; or replication delay.

- **Fix:** Confirm a **per-platform** trusted certificate profile with the correct `.cer` and store,

assigned to `SG-CPKI-Pilot`. Force a device sync; allow replication time. An intermediate needs its own trusted profile.

D. SCEP certificate not issued

- **Cause:** NDES/SCEP URL unreachable, connector SCEP feature off, template EKU/permissions wrong,

root not referenced, or an **unknown extension OID** tripped the SCEP validation allow-list.

- **Fix:** Verify the published **NDES/SCEP URL** from a device; confirm the connector's **SCEP**

feature and health; match the SCEP profile to the AD CS template (EKU = Client Auth, service account Enroll rights); reference `Trust-Root-CA`; **remove any custom extension OIDs** (only the allow-listed extensions are honored on connector v6.2510.3.2002+). Connector **Event ID 4007** = SCEP issue failure — read the event detail.

E. PKCS certificate not issued

- **Cause:** Connector PKCS feature off, CA name / template name mismatch, or service-account enroll

rights missing.

- **Fix:** Confirm the connector **PKCS** feature; enter the **exact** CA FQDN, CA name, and template

name in the profile; grant the service account **Enroll** on the template. Connector **Event ID 1052** = PKCS issue failure.

F. Wi-Fi / VPN authentication fails despite a valid certificate

- **Cause:** The Wi-Fi/VPN profile references the wrong client-cert source or trusted root; wrong EKU;

or the RADIUS/VPN server does not trust the chain.

- **Fix:** In the Wi-Fi/VPN profile, select `SCEP-Device-ClientAuth` (or `PKCS-User-ClientAuth`) as

the client certificate and `Trust-Root-CA` as the trusted root; confirm the certificate EKU is **Client Authentication**; ensure the network access server trusts the same CA chain.

G. Renewal not firing

- **Cause:** Renewal threshold not set, or connector/CA unhealthy near renewal time.
- **Fix:** Set the SCEP/PKCS profile **renewal threshold** (e.g. 20% remaining); confirm connector

and CA health; verify the device is still enrolled and receiving the profile.

H. Revocation not taking effect

- **Cause:** Revocation did not reach the CA/CRL, or the relying party does not check the CRL/AIA.
- **Fix:** Confirm the connector processed revocation (**Event ID 3050** = success; **3051** =

failure); confirm the CA published an updated CRL; ensure the RADIUS/VPN/relying party consults the CRL/AIA distribution points.

I. Scripts fail or report nothing

- **Cause:** Connected to Commercial Graph, missing scopes, or insufficient role.
- **Fix:** Reconnect with `Connect-MgGraph -Environment USGov` and the scopes listed in each script's

HOW TO RUN header; confirm **Intune Administrator** (R3). Run `CPKIG-Discovery.ps1` first to confirm the environment and connectivity.

When in doubt about a GCC High specific: re-open the Microsoft *Intune Government Service overview* page and the Microsoft 365 Roadmap filtered to **GCC High**, and re-confirm the feature status (especially **Cloud PKI**) before troubleshooting further. Never assume Commercial behavior carries over.

The deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Best Practices — Cloud PKI Integration with Intune (Government / GCC High)

Field-proven guidance for the **assign-once, repeat-everywhere** certificate model on a **GCC High** tenant, built on the **supported** path because **Cloud PKI is not yet available in GCC High**.

Sovereign-cloud discipline (GCC High)

- **Bookmark the `.us` portals and never the `.com` ones.** Sign in at `login.microsoftonline.us`;

administer at `intune.microsoft.us`, `security.microsoft.us`, `entra.microsoft.us`, `portal.office365.us`. A Commercial URL in a GCC High runbook is a defect.

- **Every script targets US Gov Graph** — `Connect-MgGraph -Environment USGov` (never `Global`).

Bake this into the header of every runbook so no one connects to the wrong cloud.

- **Treat "the option isn't there" as a parity gap, not a bug.** Before opening a support case,

check `gcc-high-parity-notes.md` — if a feature (above all **Cloud PKI**) is *planned / not available*, it genuinely is not present, and the fix is to use the GA alternative, not to troubleshoot.

- **Re-verify parity before each design.** Microsoft ships to GCC High continuously; **Cloud PKI is

planned for GCC High and may become GA** — re-check the *Intune Government Service overview* and the Microsoft 365 Roadmap filtered to GCC High before assuming today's status.

Cloud PKI specifically

- **Do not design on Cloud PKI in GCC High today** — it is **planned, not available**, and **not

planned for DoD **at all. Any date you have heard is an** ETA**; treat it as such.

• **Build the profile layer so a future Cloud PKI migration is cheap.** Keep the **trusted-root** and

SCEP profiles clean and standard; when Cloud PKI ships to GCC High you re-point them at a Cloud PKI issuing CA and retire NDES + the connector, rather than rebuilding.

• **When Cloud PKI does arrive, migrate as its own Assess→…→Expand cycle** — verify GA in *your*

tenant, pilot the issuing-CA swap, then decommission the on-prem back-end.

Certificate architecture (connector model)

• **Keep the trust chain minimal and explicit** — one `Trust-Root-CA` profile per platform (plus an

intermediate profile if needed). Every SCEP/PKCS profile references it.

• **Match the profile to the AD CS template exactly** — subject/SAN, key usage, EKU (Client

Authentication for Wi-Fi/VPN), validity, and renewal threshold must mirror the template, or issuance/renewal silently misbehaves.

• **Keep SCEP extensions standard** — the SCEP validation service **blocks unknown extension OIDs**

(connector v6.2510.3.2002+). Do not add custom OIDs to SCEP profiles.

• **Name every object verbatim and reuse it** — `Trust-Root-CA`, `SCEP-Device-ClientAuth`,

`PKCS-User-ClientAuth`, `SG-CPKI-Pilot`. Consistent names make the scripts, reports, and handover legible.

• **Prefer SCEP for device certs at scale** (key stays on-device; per-device enrollment) and **PKCS

for user certs / S/MIME** (imported PFX for S/MIME). Choose per scenario, not by habit.

Connector & NDES health

• **Keep the Certificate Connector on a supported version.** Each release is supported for **six

months**; out-of-support connectors show** Warning→Error **and can stop issuing/revoking. Allow** auto-update** (`autoupdate.msappproxy.net:443`) or plan manual updates within the window.

• **Watch the connector Event logs** (Applications and Services → Microsoft → Intune → Certificate

Connectors) — success/failure event IDs per flow (SCEP 4xxx, PKCS 1xxx, Revoke 3xxx, Health 5xxx).

• **Verify partner-CA support for GCC High** before relying on a non-Microsoft CA connector — it may

be Commercial-only.

• **Protect NDES** — publish it through your Gov reverse proxy, not directly; keep the service

account least-privileged with Enroll rights only.

Rollout & operations

• **Always pilot first, expand group-by-group.** Certificate/Wi-Fi/VPN changes can take users

offline; scope to `SG-CPKI-Pilot`, validate, then widen. Never assign tenant-wide on day one.

• **Every certificate/auth change is a change ticket** — approver, maintenance window, named pilot, documented rollback (per `global-requirements-security.md` G8).

• **Run the read-only scripts on a schedule** — `CPKIG-Discovery.ps1`, `CPKIG-Validation.ps1`, `CPKIG-Report.ps1` — to catch drift (an unassigned profile, an unhealthy connector, expiring certificates).

• **Keep change-tools in `-WhatIf` by default** — `CPKIG-Remediation.ps1` and `CPKIG-Set-ScopeTargeting.ps1` preview first; only run with `-Execute` / `-WhatIf:$false` after review and approval.

• **Least privilege:** author with **Intune Administrator** (or the custom certificate role), not Global Administrator.

The deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Deliverables Summary — Cloud PKI Integration with Intune (Government / GCC High)

This solution deploys a **managed certificate program** on a **US sovereign-cloud (GCC High)** tenant, correctly — right portals, right endpoints, and the decisive Commercial-vs-GCC-High parity gap (**Cloud PKI is not available in GCC High**) handled by building on the **supported** path: **trusted-root + SCEP/PKCS certificate profiles via the Intune Certificate Connector**, anchored to your **AD CS** CA. Proven on a pilot before scale, with a clean forward road to Cloud PKI when it ships to GCC High.

Documents

File	Purpose
`01-solution-overview.md`	The certificate model on GCC High; **Cloud PKI availability caveat up front**; how the sovereign cloud differs (portals, endpoints, parity); the Assess→Configure→Pilot→Validate→Expand model. Includes a **Commercial vs GCC High** subsection.
`02-requirements.md`	Each prerequisite as **Description / Why / Verify / Implement-if-missing** — Cloud-PKI-out-of-scope acknowledgement, GCC High tenant, Gov G3/G5 (Intune Plan 1), Intune roles, AD CS CA, NDES, the Certificate Connector, partner-CA GCC High support, endpoint allow-list, parity confirmation.
`03-implementation.md`	Exact ordered steps in the GCC High portals: trust chain, SCEP profile, PKCS profile, Wi-Fi/VPN consumption, renewal/revocation — with GCC-High URLs and the Cloud-PKI caveat + connector caveats inline where they matter, plus the future Cloud PKI migration path.

File	Purpose
`05-testing.md`	Pass/fail gates from sovereign-cloud sanity and Cloud-PKI-parity acknowledgement through issuance, consumption, renewal, revocation, and script validation.
`06-troubleshooting.md`	GCC-High-specific gotchas (Cloud PKI not present, wrong portal/cloud, endpoint blocked, connector/NDES/SCEP failures) plus common certificate failures.
`07-best-practices.md`	The assign-once, repeat-everywhere certificate model, tuned for GCC High and the connector back-end.
`gcc-high-parity-notes.md`	Reference: per-feature Commercial-vs-GCC-High status with Microsoft citations — headlined by the **Cloud PKI not-available-in-GCC-High / not-planned-for-DoD** finding.
`global-requirements-security.md`	Category-level prerequisites shared by all security solutions.
`executive-summary.md`	One-page summary for decision-makers (leads with the Cloud PKI availability caveat).

Scripts (PowerShell, Microsoft Graph SDK, US Gov endpoints)

All scripts connect with `Connect-MgGraph -Environment USGov` for GCC High. Read-only tools are read-only; change tools default to `-WhatIf`.

Script	Role	Default
`CPKIG-Discovery.ps1`	Inventory certificate connectors, trusted-root profiles, and SCEP/PKCS profiles in the GCC High tenant; confirm Cloud PKI is not in use.	Read-only
`CPKIG-Validation.ps1`	Check that the named objects exist and are assigned to the pilot group, and that the connector is healthy; flag gaps.	Read-only
`CPKIG-Report.ps1`	Certificate coverage / connector-health / profile-assignment report for review.	Read-only
`CPKIG-Remediation.ps1`	Safely remediate detected gaps (e.g. create the pilot group).	`-WhatIf`

Script	Role	Default
`CPKIG-Set-ScopeTargeting.ps1`	Build/adjust the pilot-scope group (`SG-CPKI-Pilot`) membership and profile targeting for staged expansion.	`-WhatIf`

Named objects (created and reused verbatim)

`SG-CPKI-Pilot` · trusted profile `Trust-Root-CA` · SCEP profile `SCEP-Device-ClientAuth` · PKCS profile `PKCS-User-ClientAuth` · connector reference `CertConnector-01`.

Outcome

A GCC High tenant issuing, trusting, renewing, and revoking device/user certificates through Intune on the **supported** SCEP/PKCS-via-connector path — with the Cloud PKI parity gap explicitly handled, read-only scripts to detect drift, and a documented migration to Cloud PKI once Microsoft ships it to GCC High.

The deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CPKIG-Discovery.ps1

Purpose: Connects to the US Gov Microsoft Graph endpoint for a GCC High tenant and inventories the certificate-management posture: Intune Certificate Connectors, trusted-certificate profiles, and SCEP/PKCS certificate profiles. It also confirms Cloud PKI is not being assumed — Cloud PKI is not available in GCC High (planned) and not planned for DoD, so this solution uses the connector-backed SCEP/PKCS path. Read-only — it changes nothing. Replace the sample group name your-pilot-group-name if you use a different pilot group. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CPKIG-Discovery.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\CPKIG-Discovery.ps1

CPKIG-Remediation.ps1

Purpose: Remediates the specific, safe gaps this solution can fix on a GCC High tenant: creating the pilot group if it is missing, and confirming the pilot-scope assignment of the certificate profiles. It DEFAULTS TO -WhatIf (preview only) and makes no change until you re-run with -WhatIf:$false. It never installs a connector, never issues or revokes certificates, never assigns a profile tenant-wide, and never deletes anything — those are change-controlled human actions. Replace the default names only if your objects are named differently. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CPKIG-Remediation.ps1 Apply (after review + change approval): powershell -ExecutionPolicy Bypass -File .\CPKIG-Remediation.ps1 -WhatIf:$false

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CPKIG-Remediation.ps1 Apply (after review + change approval): powershell -ExecutionPolicy Bypass -File .\CPKIG-Remediation.ps1 -WhatIf:$false

CPKIG-Report.ps1

Purpose: Generates a review-friendly report of the certificate-management posture on a GCC High tenant: the certificate connectors and their health/version, the trusted-root and SCEP/PKCS certificate profiles and their assignments, and pointers to the issued-certificate data an admin should review before expanding. Read-only — it collects and prints, it never changes configuration. Writes an optional CSV summary to the path you pass in OutputCsv. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CPKIG-Report.ps1 powershell -ExecutionPolicy Bypass -File .\CPKIG-Report.ps1 -OutputCsv .\cpkig-report.csv

How to run: powershell -ExecutionPolicy Bypass -File .\CPKIG-Report.ps1 powershell -ExecutionPolicy Bypass -File .\CPKIG-Report.ps1 -OutputCsv .\cpkig-report.csv

CPKIG-Set-ScopeTargeting.ps1

Purpose: Manages membership of the certificate scope group (SG-CPKI-Pilot by default) so you can stage a certificate rollout from pilot to production a batch at a time. It adds the users or devices you pass in AddMembers to the scope group. It DEFAULTS TO -WhatIf (preview only) and changes nothing until you re-run with -WhatIf:$false. It does not remove members and does not retarget profiles directly —

expansion stays deliberate and reversible. Pass member object ids or user principal names such as pilot-user@your-agency.gov (no angle brackets). HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\CPKIG-Set-ScopeTargeting.ps1 -AddMembers user1@your-agency.gov,user2@your-agency.gov Apply (after review + change approval): powershell -ExecutionPolicy Bypass -File .\CPKIG-Set-ScopeTargeting.ps1 -AddMembers user1@your-agency.gov -WhatIf:$false

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\CPKIG-Set-ScopeTargeting.ps1 -AddMembers user1@your-agency.gov,user2@your-agency.gov Apply (after review + change approval): powershell -ExecutionPolicy Bypass -File .\CPKIG-Set-ScopeTargeting.ps1 -AddMembers user1@your-agency.gov -WhatIf:$false

CPKIG-Validation.ps1

Purpose: Checks a GCC High tenant for the specific, named objects this solution creates and reports PASS or FAIL for each: the pilot group, the trusted-root profile, the SCEP profile, the PKCS profile, and at least one healthy Intune Certificate Connector. It does not change anything. Use it as the pilot validation gate before expanding scope. Cloud PKI is not available in GCC High, so validation targets the connector-backed SCEP/PKCS path. Replace the default names only if you named your objects differently. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CPKIG-Validation.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\CPKIG-Validation.ps1